

Manual Operasional — QR Code Security System (RSA-PSS)

Panduan menjalankan sistem, mengeceknya, serta memantau & menghentikan proses. Server: Ubuntu, aplikasi berjalan via **systemd** (`qrcode.service`).

1. Arsitektur Singkat

```
Internet —> Nginx (443/80) —> Gunicorn (127.0.0.1:5000) —> Aplikasi Flask (wsgi:app)
                                     |
                                     — Redis (rate limit / state)
```

Komponen	Service systemd	Peran
Aplikasi	<code>qrcode.service</code>	Gunicorn menjalankan <code>wsgi:app</code> (1 worker, 12 threads, port 5000)
Reverse proxy	<code>nginx.service</code>	Meneruskan <code>https://rsa-pss.com</code> → <code>127.0.0.1:5000</code>
Cache/state	<code>redis-server.service</code>	Dependensi aplikasi

PENTING: Aplikasi **HARUS** dijalankan lewat systemd. **JANGAN** menjalankan `python app.py` manual dari terminal — bila terminalnya ditutup, stdout menjadi tidak valid dan proses seperti kalibrasi gagal dengan `[Errno 5] Input/output error`.

2. Menjalankan & Mengelola Sistem

Semua perintah butuh `sudo`.

```
# Menyalakan aplikasi
sudo systemctl start qrcode

# Mematikan aplikasi
sudo systemctl stop qrcode

# Restart (mis. setelah update kode/template)
sudo systemctl restart qrcode

# Lihat status ringkas
sudo systemctl status qrcode

# Aktifkan auto-start saat server booting (sudah aktif)
sudo systemctl enable qrcode
```

```
# Matikan auto-start saat booting
sudo systemctl disable qrcode
```

Setelah **reboot server**, aplikasi + redis + nginx menyala otomatis (semua sudah **enabled**). Jika aplikasi crash saat berjalan, systemd otomatis menghidupkannya lagi (**Restart=always**, jeda 5 detik).

3. Melihat Log

```
# Ikuti log aplikasi secara real-time (Ctrl+C untuk keluar)
sudo journalctl -u qrcode -f

# Log 30 menit terakhir
sudo journalctl -u qrcode --since "-30 min" --no-pager

# Cari error tertentu
sudo journalctl -u qrcode --no-pager | grep -i "error\|traceback"
```

Log tambahan di dalam folder aplikasi: **logs/app.log**, **logs/server.log**.

4. Cek Sistem Berjalan & Sehat

```
# 1) Service aktif?
sudo systemctl is-active qrcode          # harus: active

# 2) Proses gunicorn ada?
pgrep -fa "gunicorn.*wsgi:app"

# 3) Port 5000 mendengarkan?
sudo ss -tlnp | grep :5000

# 4) Aplikasi merespons? (harus 200)
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://127.0.0.1:5000/testing/calibration

# 5) Situs publik hidup? (harus 200/302)
curl -s -o /dev/null -w "HTTP %{http_code}\n" https://rsa-pss.com/
```

Cek stdout sehat (mencegah bug **[Errno 5]**) — harus menunjuk **socket:[...]**, **bukan pts (deleted)**:

```
sudo ls -l /proc/$(pgrep -f "gunicorn.*wsgi:app" | head -1)/fd/1
```

5. Cek Proses Background (Kalibrasi / Generate Massal / Verifikasi Massal)

Proses berat berjalan sebagai **thread di server**, bukan di browser. Menutup browser **tidak** menghentikannya — pekerjaan lanjut sampai selesai.

5a. Lewat Browser (cara utama)

Buka kembali halaman terkait; progress bar akan otomatis memuat status terkini dari server (via `task_id`). Halaman & endpoint pemantauan:

Proses	Halaman	Endpoint progress
Kalibrasi	<code>/testing/calibration</code>	<code>/testing/calibration_progress</code>
Generate massal	halaman generate	<code>/api/generate_progress_status</code>
Verifikasi massal	halaman verifikasi massal	<code>/api/verify_massal_progress_status</code>

5b. Lewat Server (cek "apakah ada batch yang sedang jalan")

Batch yang aktif membuat worker unicorn sibuk (CPU tinggi). Ukur **CPU sesaat**:

```
CLK=$(getconf CLK_TCK)
read_ticks(){ local t=0; for p in $(pgrep -f "unicorn.*wsgi:app"); do read -ra S < /proc/$p/stat; t=$((t+${S[13]}+${S[14]})); done; echo $t; }
a=$(read_ticks); sleep 3; b=$(read_ticks)
echo "CPU sesaat worker: $(( (b-a)*100/(CLK*3) ))%"
```

- **~0%** → tidak ada batch berat yang jalan (idle).
- **80-100%** → ada proses (kalibrasi/generate/verify) sedang berjalan.

Catatan: `ps aux` menampilkan **rata-rata CPU seumur proses**, bukan sesaat — kurang akurat untuk menilai idle. Gunakan cara di atas.

6. Menghentikan Proses Background

1. **Cara aman (disarankan):** tekan tombol **Stop** di halaman prosesnya. Endpoint yang dipakai UI: `stop_calibration`, `/api/stop_generate_process`, `/api/stop_verify_massal_process`.
2. **Cara paksa:** `sudo systemctl restart qrcode` ⚠ Ini mematikan **SEMUA** proses background yang sedang jalan (kalibrasi/batch) di tengah jalan dan **tidak** dilanjutkan otomatis. Pakai hanya saat darurat atau saat yakin tidak ada batch penting berjalan (cek dulu dengan langkah 5b).

7. Stress Testing (Simulated vs Real HTTP)

Sistem punya **dua** mode uji beban yang menjawab pertanyaan **berbeda**. Keduanya bukan pembandingan langsung.

	Simulated Stress Testing	Real HTTP Stress Test
Cara kerja	Model analitis di memori	Request HTTP sungguhan ke aplikasi
Jaringan	Tidak ada	Ada (koneksi nyata)
Rate limit	Tidak kena	Kena (kecuali beban lokal, lihat bawah)
Artefak	Tidak membuat file	Membuat QR, menulis log generate/verifikasi
Level user wajar	100 – 1.500	Bergantung endpoint (lihat 7b)

7a. Kenapa level user Real HTTP lebih kecil

Karena dibatasi kapasitas nyata server, bukan sekadar angka di form:

- **CPU 2 core**, Gunicorn **1 worker × 12 threads**.
- Generator beban berjalan **di dalam proses aplikasi yang sama**, sehingga penembak dan yang ditembak berebut CPU yang sama.
- Di atas ±100 user, yang terukur adalah **antrean dan timeout**, bukan performa kriptografi RSA-PSS.

Workers wajib tetap 1. `background_tasks` (`app.py`) dan `_calibration_state` (`routes/testing_routes.py`) disimpan di memori proses. Menambah worker membuat polling progres mendarat di worker yang tidak mengenal task tersebut → progres task/kalibrasi rusak. Untuk menambah kapasitas, naikan **threads** (`GUNICORN_THREADS`), bukan workers.

7b. Kapasitas terukur (hasil pengukuran nyata di server ini)

Angka berikut diukur langsung, bukan perkiraan. **Level user yang wajar sangat bergantung pada endpoint yang dipilih.**

Generate + Verify QR (berat: RSA-PSS + render QR + tulis file & log):

Users	Sukses	Avg response	Throughput
5	100%	17,9 s	~0,28 alur/s
10	100%	36,5 s	~0,27 alur/s
25	100%	77,9 s	~0,32 alur/s

Throughput **mendatar di ~0,3 alur/detik** — sistem sudah jenuh sejak 5 user. Menambah user **tidak** menambah throughput, hanya melipatgandakan latensi. Karena itu level di atas 25 tidak berguna untuk endpoint ini.

Server Metrics / Dashboard (ringan: tanpa kriptografi & tanpa tulis file):

Users	Sukses	Avg response	Throughput
25	100%	82,9 ms	~302 ops/s
50	100%	153,1 ms	~327 ops/s
100	100%	264,7 ms	~378 ops/s

Di sini level 100 user berjalan mulus tanpa satu pun HTTP 429. Jadi bila ingin menguji konkurensi tinggi, gunakan endpoint ringan.

Timeout wajib besar untuk Generate + Verify. Pada 25 user satu alur butuh ~78 detik. Dengan timeout kecil (mis. 15 detik) hampir semua request gagal sebagai timeout dan dilaporkan sebagai **status 0** — terlihat seperti "throughput 0 dan HTTP 429 juga 0". Default sekarang: **120 detik**.

7c. Aturan penting: Requests per User Level

Konkurensi nyata per level = **min(concurrent users, operations)**.

Artinya **Requests per User Level harus $\geq$ level user tertinggi**, kalau tidak level tinggi tidak pernah benar-benar tercapai.

Level user	Requests per level	Konkurensi nyata
100	20	20 X (level 100 tidak tercapai)
100	200	100 ✓

Nilai default sekarang: level **5,10,25** dengan **30** requests per level dan timeout **120** detik.

7d. Base URL: lokal vs publik

Base URL	Efek
http://127.0.0.1:5000/ (default)	Langsung ke Unicorn. Rate limit dikecualikan , pengguna publik tidak terganggu. Dipakai untuk level user tinggi.
https://rsa-pss.com/	Menyertakan overhead Nginx/HTTPS, tetapi rate limit 60/menit aktif → banyak HTTP 429.

Pengecualian rate limit hanya berlaku bila request memenuhi **dua** syarat sekaligus: ber- **User-Agent: QRRealHTTPStress/1.0** **dan** berasal dari **127.0.0.1**. Klien dari luar tidak bisa memalsukannya, karena Nginx selalu menimpa **X-Forwarded-For** dengan IP asli pemanggil.

Keterbatasan pada Generate + Verify. URL verifikasi yang ditanam di dalam QR selalu memakai **BASE_URL** dari **.env** (**https://rsa-pss.com/**). Jadi tahap verify tetap melewati Nginx/HTTPS publik walaupun Base URL diisi localhost, sehingga tahap itu **tidak** ikut dikecualikan dari rate limit. Hanya tahap generate yang memakai jalur lokal.

Catatan cookie (penyebab lama status 0). Cookie sesi ber-flag `Secure` karena `REQUIRE_HTTPS=true`. Karena itu proses login penembak beban sengaja **tidak mengikuti redirect** dan mengambil `Set-Cookie` langsung dari respons login. Bila redirect diikuti pada jalur HTTP lokal, cookie login yang valid tertimpa cookie "belum login" sehingga seluruh request menjadi anonim dan dilaporkan sebagai status 0.

7e. Catatan untuk penelitian

Sajikan kedua mode sebagai pengukuran yang berbeda, bukan perbandingan angka langsung:

- **Simulated** → model skala besar (100–1.500 user).
- **Real HTTP** → validasi empiris end-to-end pada kapasitas server nyata.

Bila butuh konkurensi tinggi yang sah secara metodologis, beban harus ditembakkan dari **mesin terpisah** (mis. k6/locust dari host lain), bukan dari server yang sama.

8. Peringatan Penting

- ✗ **Jangan** `python app.py` manual → memicu `[Errno 5] Input/output error` pada kalibrasi.
- ⚠ **Restart service saat batch jalan** → batch berhenti di tengah, tidak resume. Selalu cek CPU (bagian 5b) sebelum restart.
- ⚠ **Setelah mengubah kode/template**, wajib `sudo systemctl restart qrcode` (template di-cache di memori karena `debug=False`).
- ● Service berjalan sebagai **root** (mengikuti kepemilikan file `data/`, `logs/`, DB).

9. Troubleshooting Cepat

Gejala	Kemungkinan sebab	Tindakan
Situs tidak bisa diakses	qrcode/nginx mati	<code>sudo systemctl status qrcode nginx</code> lalu <code>start</code>
HTTP 502 di browser	gunicorn mati / port 5000 kosong	<code>sudo systemctl restart qrcode</code> ; cek <code>ss -tlnp grep :5000</code>
Kalibrasi gagal <code>[Errno 5]</code>	app dijalankan manual (stdout mati)	Pastikan lewat systemd; cek fd (bagian 4)
Perubahan kode tak muncul	template/kode ter-cache	<code>sudo systemctl restart qrcode</code>
Ingin tahu sebab error	—	<code>sudo journalctl -u qrcode --since "-15 min"</code>

10. Ringkasan Perintah (Cheat Sheet)

```
sudo systemctl start   qrcode    # nyalakan
sudo systemctl stop    qrcode    # matikan
sudo systemctl restart qrcode    # restart
sudo systemctl status  qrcode    # status
sudo journalctl -u qrcode -f      # log real-time
pgrep -fa "unicorn.*wsgi:app"    # cek proses
sudo ss -tlnp | grep :5000       # cek port
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:5000/testing/calibration
```